

Lifelong Learning for a Kresling-Origami Robot: A Plug-and-play Module for Supervised Learning of Target Adaptations

E. De Vroey

Abstract—...

I. INTRODUCTION

II. BACKGROUND

A. Lifelong Learning

B. Kresling Origami

III. METHODOLOGY

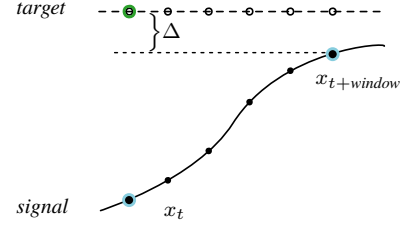
A. Target Adaptations

The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

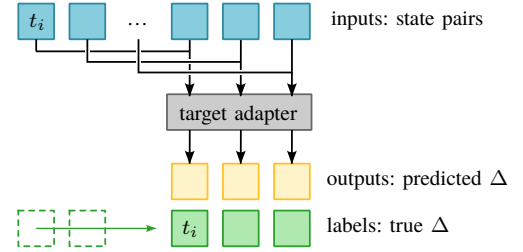
The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state $\mathbf{x}_d$. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, the adapter module *lies* to the controller about reaching $\mathbf{x}_d$ and instead provides it with $\mathbf{x}_d + \Delta$, for which the controller yields a larger action u^* that *does* result in the system reaching state $\mathbf{x}_d$.

The *adapter* module is implemented as a feedforward neural network with two hidden layers with non-linear activations. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. The model is then provided with pairs of system states, spaced by a certain time window $\mathbf{x}_t$ and $\mathbf{x}_{t+window}$, that are obtained from these recordings. From these pairs, the model infers the target it 'believes' the controller aims to reach. However, instead of predicting the target directly, the model learns the difference Δ between the target and the latter state in the pair (Figures 1a and 1b).

The model can be trained in a supervised manner (Figure 1b) and then deployed. During deployment, it is presented with the current state $\mathbf{x}_t$ but cannot be presented with $\mathbf{x}_{t+window}$ (since this state is not known yet). Instead, the second state in



(a) The target adaptation problem: Based on the current position x_t and the position some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



(b) The training process of the target adapter: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a Δ , being the difference between the target $\mathbf{x}_d$ and state $\mathbf{x}_{t+window}$.

Fig. 1: Target adaptation during the training phase

the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a Δ that, when added to the latter state in the pair (i.e., the desired target), results in an adapted target (Figure 2a) for which the controller will produce a control action. In short, given a current state $\mathbf{x}_t$ and a future state $\mathbf{x}_{t+window}$, the model learned to produce the desired target. Thus, when presented with $\mathbf{x}_t$ and $\mathbf{x}_{t+window} = \mathbf{x}_d$ (supposedly reached within the time window) it provides the (adapted) target that would “push” the controller toward that result. A full overview is presented in Figures 2a and 2b.

The reason for predicting the difference Δ between the state $\mathbf{x}_{t+window}$ and the desired target, rather than a direct prediction of the target, can also be clarified now: By designing the model's output to be the difference Δ , the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.

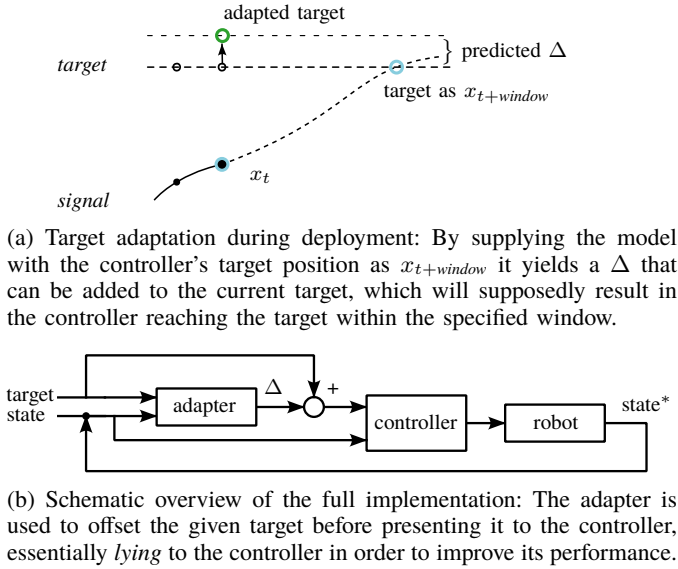


Fig. 2: Target adaptation during deployment.

B. Toy Problem

To explore the proposed method, a toy problem is considered first. The toy problem consists of a simulation of a simple 1D mass-spring-damper system (Figure 3) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

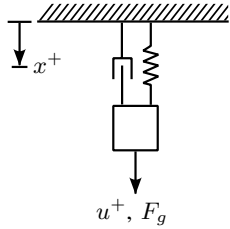


Fig. 3: Setup of the mass-spring-damper system in the toy problem with positive directions indicated.

The system state $\mathbf{x}$ consists of the position and velocity $x, \dot{x}$, respectively. The controller aims to reach targets of the form $\mathbf{x}_d = [x_d, 0]$, but controls position only. At regular intervals, a random target position x_d is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required Δ , and the controller computes the control action based on the current position and the adapted target position. The system's response to this control input is simulated and the tuple $(x_0, \dot{x}_0, u, x, \dot{x}, x_d + \Delta, 0)$ is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data. Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. It is important that the tuples in the training data always contain the target position that was actually provided to the controller, i.e., $x_d + \Delta$, and not the *true* desired target x_d . As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation.

The pseudo algorithm for the implementation can be found in Algorithm 1.

Algorithm 1: Toy problem target adaptation

Data: system state $\mathbf{x}$, target $\mathbf{x}_d$, adaption Δ , control action u

Initialize model: *adapter*;
 Initialize buffer: $buffer \leftarrow []$;
for each timestep t_i **do**
 if $len(buffer) > threshold$ **then**
 Remove the oldest element in the buffer;
 end
 if $t_i \bmod target_interval = 0$ **then**
 $\mathbf{x}_d \leftarrow random_target()$;
 end
 if $t_i \bmod update_interval = 0$ **then**
 $(features, labels) \leftarrow prep_data(buffer, window)$;
 $adapter.fit(features, labels)$;
 end
 $\Delta \leftarrow adapter.predict([\mathbf{x}_0, \mathbf{x}_d])$;
 $u \leftarrow controller.compute_control(\mathbf{x}_0, \mathbf{x}_d + [\Delta, 0])$;
 Simulate the system response $\mathbf{x}$;
 Update system parameters;
 Append $(\mathbf{x}_0, u, \mathbf{x}, \mathbf{x}_d + [\Delta, 0])$ to $buffer$;
 $\mathbf{x}_0 \leftarrow \mathbf{x}$;
end

C. Preliminary Results

The toy problem was implemented in Python: simulating the system response using SciPy's `lsim`, and implementing the adapter in Keras with the PyTorch backend. The simulation ran for 60 s at 50 Hz. The target was changed every 5 s, and the adapter was trained fully online, updating every 15 s. Training features were created for multiple prediction windows, this means the training features consisted of pairs $[\mathbf{x}_t, \mathbf{x}_{t+window}]$ for windows of 3, 5, 10, 15, and 25. The buffer's budget corresponded to the past 30 s of data. For reference, a second controller and system were also simulated, identical to the described simulation, but without the addition of an adapter. To both the reference and adapted signal, a small amount of Gaussian noise was added to simulate measurement noise.

1) *A case where the controller is too aggressive:* For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high. In Figure 4, the result of the simulation can be observed. At the start, and due to the near-zero initialization of the adapter's parameters, the response signal of the system controlled by the adapted controller is indistinguishable from that of the reference controller. However, as soon as the adapter has trained on the buffer, the changes to the target become visible.

The effects of the adaptation can be more clearly observed in Figure 5. Since the reference controller is too aggressive, the reference signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots,

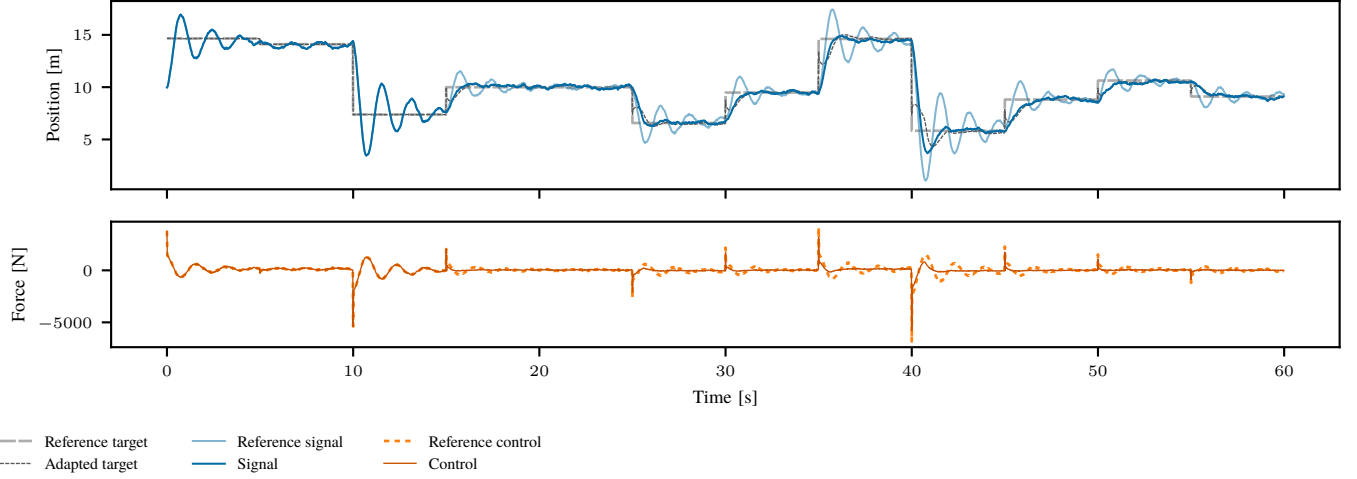


Fig. 4: The full minute simulation of the toy problem with an aggressive controller. Top: The system response x and target $x_d + \Delta$ of the adapted controller, alongside the response and target of the unadapted reference controller ($\Delta = 0$). Bottom: The resulting adapted control inputs from the controller receiving $x_d + \Delta$ alongside the unaltered reference control inputs. During the first 15 s (3 targets) the adapter has not received updates yet, and the signal is thus identical to the reference signal.

where the adapter would often encounter training features for which the labeled target position¹ x_d is somewhere in between x_t and $x_{t+window}$. Therefore, upon receiving the input feature $[\mathbf{x}_t, \mathbf{x}_d]$ it produces an adapted target position in between x_t and x_d . As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

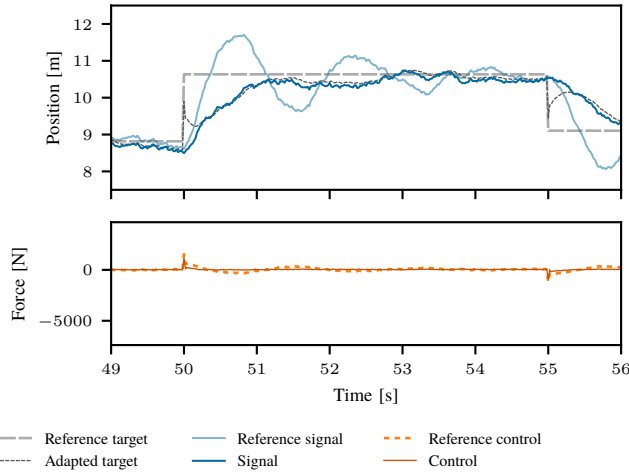


Fig. 5: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the reference signal) is countered by bringing the target closer to the current state.

2) *A case where the controller is under-responsive:* For a second simulation, the system and controller were designed

¹Rather, the labeled $\Delta = x_d - x_{t+window}$, but the explanation provided is equivalent and somewhat clearer.

such that the controller was more *conservative* or *under-responsive*. Additionally, the signal produced by the reference controller has a clear steady-state error. The result of the simulation can be found in Figure 6.

Once more, the adapter is able to improve the controller's performance after just a single update. In Figure 7 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs $\mathbf{x}_t, \mathbf{x}_{t+window}$ where the target position x_d is positioned further from x_t than it is from $x_{t+window}$. As a result, the adapter learns to predict adaptations that would move x_d farther from x_t , making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs $\mathbf{x}_t, \mathbf{x}_{t+window} = \mathbf{x}_t$, which appear to represent the target state being reached, even though $x_{t+window} \neq x_d$. These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

IV. A KRESLING ORIGAMI ROBOT

A. Mechanical Details

As mentioned in Section II, origami robots—and by extension, soft robots—are a suitable use-case for lifelong learning techniques. To evaluate the method described in Section III, a robot was designed that serves as a good analogue to the toy problem, whilst also using kresling origami as a key part of its functionality. Figure 8 provides important nomenclature for reference.

The robot is a simple crawler consisting of two Kresling origami springs (KOS) (Figure 8a, item 1) that are mirror images of each other, connected in series, with the actuation mechanism in between. Both springs can expand and contract in sync, producing a crawling-like motion when combined with

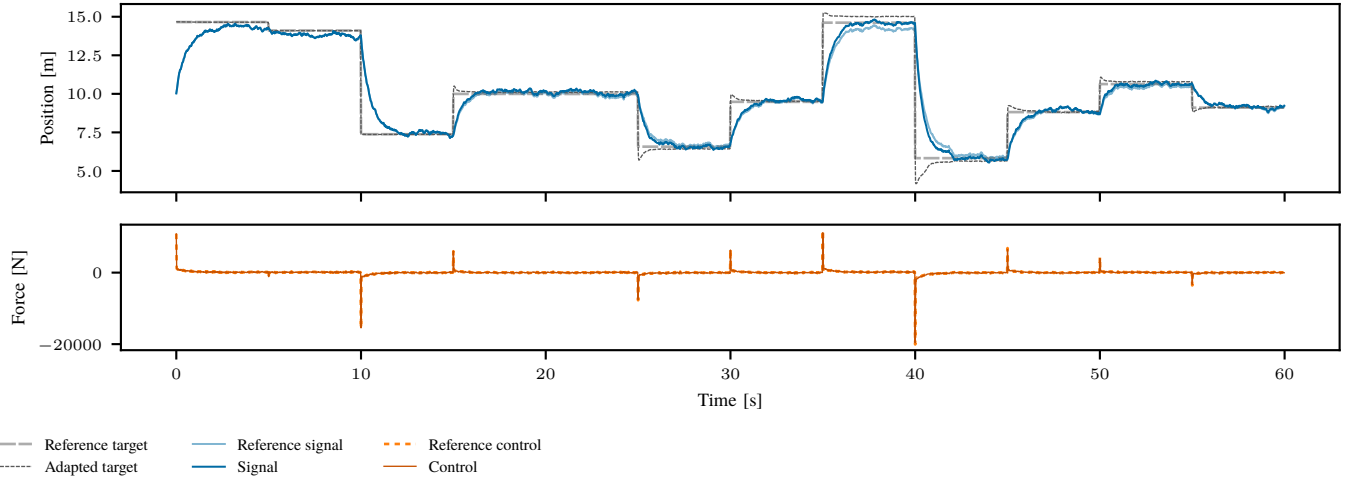


Fig. 6: The full minute simulation of the system with an under-responsive controller. Top: The system response and target of the adapted controller, alongside the response and target of the unadapted reference controller. Bottom: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered reference control inputs.

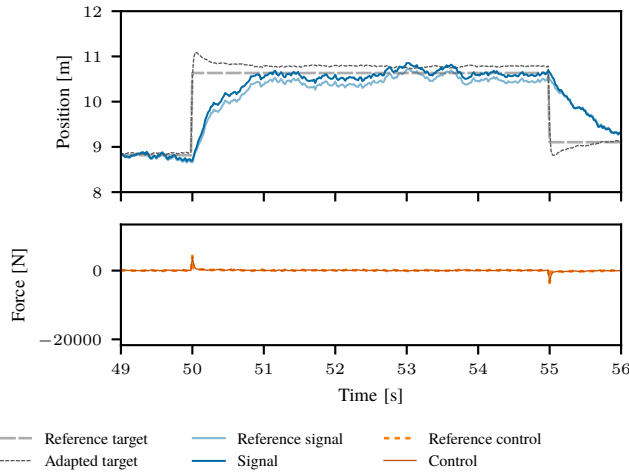


Fig. 7: A close-up of the second to last target in the one-minute simulation. Notable observations: The target is initially moved further from the current state to ‘pull’ it towards the desired state in a shorter period. In addition, the adapted target is offset from the desired state as to counter the steady-state error seen in the reference signal.

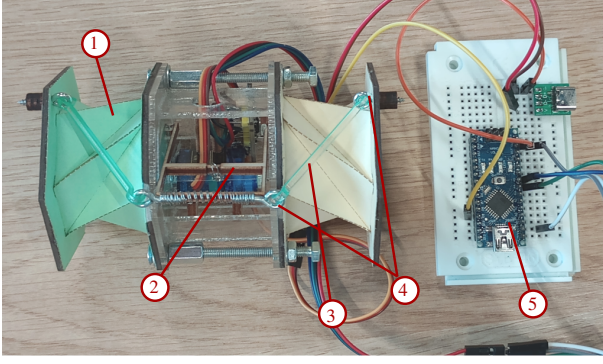
specially designed “feet”. To demonstrate target adaptation, however, the movement of the robot as a whole is not very important. Rather, the state of the KOSs is the state that will be controlled.

A KOS can be actuated both by linear motion (pushing or pulling on the spring) and by rotational motion (twisting). Since the two motions are directly related (Section II-B), a contracting or expanding motion can be measured as the angle of the KOS’ twist. The twist is induced by turning a rotating arm (2) which is connected to the ends of the KOSs by two

links (3) that hook into eye-screws (4). To turn the arm, a micro servo motor (6) is employed. In the toy problem, the controller yields a force to exert on the system in order to reach the target position. However, most micro servo’s come with a near perfect position controller built-in, which cannot be accessed or changed. Therefore, to bridge the gap between the simulation and the real-world implementation, the position output of the servo is converted to a force output by the use of a small, metal spring (8). The servo thus does not directly act on the actuation arm, but rotates a servo arm (7) instead, which pulls the spring and thereby exerts a force on the actuation arm. As previously mentioned, the state of the KOS can be measured as the angle of twist, since it is directly related to its contraction. Likewise, assuming perfectly rigid links with negligible clearance between the hooks and eye-screws, the angle of twist is directly related to the angle of the actuation arm. In the full implementation, the robot’s state $\mathbf{x}_t$ will thus be measured as the actuation arm’s angular position and velocity $[\theta, \dot{\theta}]$. This angle is measured using a contactless sensor (9) that can measure the on-axis rotation of a small magnet (10) attached to the actuation arm. The sensor relays this information to an Arduino Nano (5) board, which runs a PID controller that returns the control action, i.e., an angular position for the servo. Furthermore, there are mechanical limits for the rotation of both the actuation arm and servo arm, imposed by the edge of the container housing the actuation mechanism and the supports holding the servo motor, respectively. However, the design is such that the servo’s mechanical limit corresponds to a θ below the actuation arm’s limit.

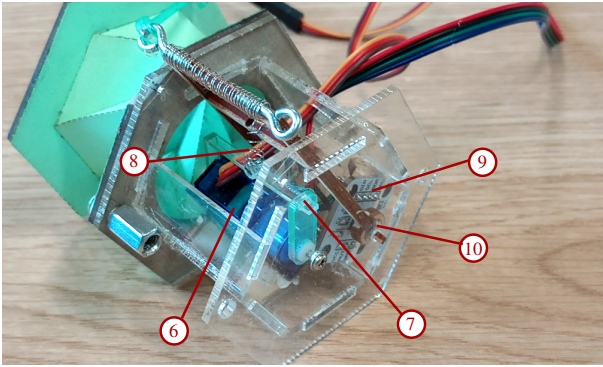
B. Software Implementation

The supervised learning implementation for target adaptation is divided into two key components: the nominal control of the robot, programmed in C++ and executed by the Arduino



- | | |
|--------------------|--------------|
| 1) KOS | 4) Eye screw |
| 2) Actuation arm | 5) Arduino |
| 3) Connecting link | |

(a) The Kresling origami robot.



- | | |
|-----------------|-----------------|
| 6) Servo motor | 9) Angle sensor |
| 7) Servo arm | 10) Magnet |
| 8) Metal spring | |

(b) Actuation mechanism.

Fig. 8: Annotated views of the robot and its most important actuation mechanisms

board, and the target adaptation, implemented in Python and managed by an external computer.

1) *Nominal Control on the Arduino:* Upon startup, the Arduino initializes a connection with the external computer and directs the servo to move to its minimum position. The Arduino allows some time for the servo to reach this home position and then reads a first measurement from the angle sensor, setting this as the reference $\theta = 0$. During the control loop, the servo reads θ (relative to the reference) and $\dot{\theta}$ received from the sensor, and reads the target θ_d received from the external computer. Based on these measurements, the PID controller computes the value to be written to the servo, but before writing it to the servo, a ceiling is imposed on it to protect the servo from exceeding its mechanical limit (Section IV-A). The Arduino allows some time for the servo to act and then reads the measurements from the angle sensor again. Finally, it sends the tuple containing the state, action, resulting state, and target to the external computer.

2) *Target Adaptation on the External Computer:* The target adaptation and supervised learning are implemented exclusively on the external computer through the established connection with the Arduino. The program is divided into several modules or *threads* that run simultaneously. This way, values that need to be sent or read can be updated in the background upon request, without delaying the operations of other modules (see Appendix A for a more detailed breakdown of the interactions between various threads). At regular predefined intervals, a `change_value` thread changes the true target θ_d to some value. Every iteration, it constructs an input feature $[\theta, \dot{\theta}, \theta_d, 0]$ for the adapter. The resulting predicted Δ is added to the target, which is read by the `send` thread and forwarded to the Arduino. The buffer is maintained by storing the tuples received from the Arduino. Upon train-time, the adapter weights are updated in the background, without interrupting the adapter's performance.

V. EXPERIMENTS AND RESULTS

A. Sign Conventions and Unit Clarification

Before the experiments and results can be properly understood, it is necessary to clarify certain sign and unit conventions. Most importantly, the contactless magnetic sensor denotes a clockwise rotation (as viewed from the top) w.r.t. the datum as a positive angle, and likewise, a counterclockwise rotation as a decrement in the angle. In practice, this means that a positive control input from the servo (i.e. a positive angle) results in a decrement in θ . Because the actuation arm's calibration point is set when the servo is at its minimum output, the operational range of the actuation arm is entirely in the negative domain. Figure 9 illustrates the servo arm and actuation arm in a configuration representative of a typical scenario during the experiments, with the signs of the angles indicated.

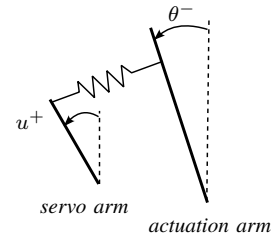


Fig. 9: Nominal signs of the angles for the servo and actuation arm during operation.

In addition, The *Servo* Arduino library uses the `writeMicroseconds(value)` function to write to the servo, which maps a servo's range to a value between 1000 and 2000 μs depending on the servo's manufacturer. As such, `writeMicroseconds` provides more 'resolution' than the common `write(angle)`, which works with integer values of angles in degrees. Note that the term *microseconds* in this context does not refer in any way to the angular unit of measurement *arcseconds*. Instead, it relates to the pulse width in microseconds of the signal that the Arduino sends to the servo motor. In this case, the servo's minimum

position corresponds to a value of $540\mu\text{s}$ and the maximum mechanical limit (as discussed in Section IV-A) corresponds to approximately $1800\mu\text{s}$. It might thus be confusing to see the control input, the angle of the servo arm, denoted in a unit of time. For convenience, the $540\mu\text{s}$ value relates to 0 degrees, and the $1800\mu\text{s}$ limit corresponds to 125 degrees.

Finally, during the experiments, the Arduino initialized a *counter* value to include in the tuple sent to the computer. This counter starts at 0 and simply increments by 1 with each iteration of the Arduino's loop. Since the interactions between the Arduino and computer, as well as between the several threads on the computer, are independent of timing (i.e., the modules do not have to "wait for each other"), the counter's value is essential to accurately plot the several signals recorded during the experiments. As a result, the counter itself does not measure elapsed time directly, making it unitless. However, it can be used as a proxy for time, since all modules aim to operate at 50 Hz. This means that an increment of 1 in the counter corresponds to an estimated 0.02 s, allowing for the approximation of elapsed time based on the counter value.

B. Performance Metrics

C. Experimental Setup

For each experiment a minimum of two sets of KOSs was required: One set serving as a reference and baseline, with the Arduino receiving unadapted targets, and the second set used for testing target adaptation. All experiments used a 60 s buffer and 15 s intervals between updates, with the first update at 60 s. The adapter trained on prediction windows of 3, 5, 10, 15, and 25 steps. The true target, with values depending on the experiment, was updated every 5 s, following a predefined random sequence. This sequence amounts to 5 min of deployment time and is repeated according to the full experiment duration. The signals corresponding to each 5 min sequence are recorded and saved for evaluation. This way, the evolution of the evaluation metrics can be studied, and a fair comparison between the reference controller and adapted controller is ensured. Figure 10 shows the general experimental setup.

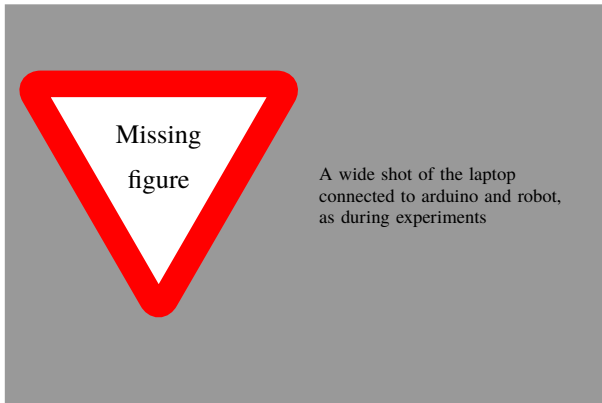


Fig. 10: The test setup.

VI. LIMITATIONS

VII. DISCUSSION

APPENDIX A CODE STRUCTURE

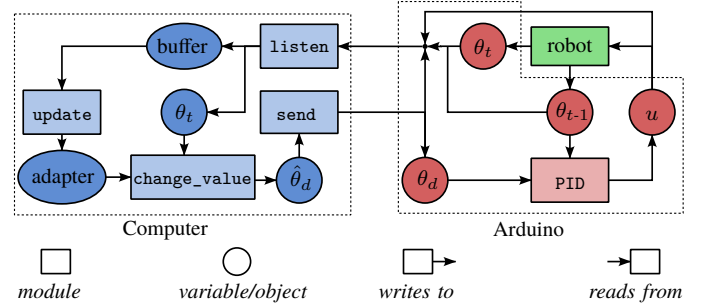


Fig. A.1: The interaction between the modules (rectangles) and shared variables (ovals) of the computer (blue) and Arduino (red).

Figure A.1 provides an overview of the communication between computer and Arduino, and their inner interactions between various modules. For example, the sole task of the *send* thread is to read the value of a global variable representing the target state and send this to the Arduino. The *listen* thread reads the tuple sent in return by the Arduino and assigns its contents to the respective shared variables for the state, control action, resulting state and target. It then adds this tuple to the buffer used for updating the adapter model. The *update* thread is responsible for (re)training the adapter. It creates a copy of the adapter, which can then be trained in the background while the original adapter remains functional. Once training is completed, the copied adapter's weights are assigned to the original adapter. The *change_value* thread is responsible for setting the "true" target in the control loop, as well as the actual adaptations to it. All threads aim to operate at 50 Hz.